🛠️ Setup e Instalação

Guia completo de configuração do projeto Lulu.

Índice

- Requisitos
- Instalação
- Configuração Firebase
- Variáveis de Ambiente
- Scripts Disponíveis
- Build e Deploy
- Troubleshooting

Requisitos

Software Necessário

Software	Versão Mínima	Recomendada
Node.js	18.x	20.x LTS
Yarn	1.22.x	Latest
Git	2.x	Latest

Verificar Versões

```
node --version    # v20.x.x
yarn --version    # 1.22.x
git --version     # 2.x.x
```

Instalar Yarn (se necessário)

```
npm install -g yarn
```

Instalação

1. Clonar o Repositório

```
git clone https://github.com/seu-usuario/lulu.git
cd lulu
```

2. Instalar Dependências

```
yarn install
```

Isso instalará todas as dependências listadas em `package.json`, incluindo:

- Next.js 15.1.3
- React 19
- Firebase 11
- TanStack React Query
- Radix UI components
- E mais...

Tempo estimado: 2-5 minutos dependendo da conexão

Configuração Firebase

1. Criar Projeto no Firebase

1. Acesse Firebase Console
2. Clique em "Adicionar projeto"
3. Nomeie o projeto (ex: "lulu-app")
4. Desabilite Google Analytics (opcional)
5. Clique em "Criar projeto"

2. Habilitar Authentication

1. No menu lateral, vá em **Authentication**
2. Clique em "Começar"
3. Habilite os provedores desejados:
    - Email/Password ☑
    - Google (opcional)
    - Outros conforme necessidade

3. Criar Firestore Database

1. No menu lateral, vá em **Firestore Database**
2. Clique em "Criar banco de dados"
3. Selecione **Modo de produção** ou **Modo de teste**
4. Escolha a localização (ex: `southamerica-east1` para São Paulo)
5. Clique em "Ativar"

4. Configurar Storage

1. No menu lateral, vá em **Storage**
2. Clique em "Começar"

3. Aceite as regras padrão
4. Escolha a mesma localização do Firestore
5. Clique em "Concluído"

5. Obter Credenciais

1. No menu lateral, vá em **Configurações do projeto** (ícone de engrenagem)
2. Na aba "Geral", role até "Seus aplicativos"
3. Clique no ícone **</>** (Web)
4. Registre o app com um apelido (ex: "lulu-web")
5. **NÃO** marque "Configure Firebase Hosting"
6. Copie as credenciais exibidas

Variáveis de Ambiente

1. Criar Arquivo `.env.local`

```
cp .env.example .env.local
```

2. Preencher Credenciais

Abra `.env.local` e adicione suas credenciais do Firebase:

```
# Firebase Configuration
NEXT_PUBLIC_FIREBASE_API_KEY=AIzaSy...
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=lulu-app.firebaseapp.com
NEXT_PUBLIC_FIREBASE_PROJECT_ID=lulu-app
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET=lulu-app.appspot.com
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=123456789012
NEXT_PUBLIC_FIREBASE_APP_ID=1:123456789012:web:abcdef...

# PWA Configuration (Opcional)
NEXT_PUBLIC_BASE_URL=http://localhost:3001  # Em desenvolvimento
# NEXT_PUBLIC_BASE_URL=https://seu-dominio.com  # Em produção

# Serwist (Opcional - suprime warning Turbopack)
SERWIST_SUPPRESS_TURBOPACK_WARNING=1
```

3. Variáveis para Produção

Adicione também um arquivo `.env.production` com as mesmas variáveis, mas com valores de produção:

```
NEXT_PUBLIC_BASE_URL=https://luluzinha.web.app
```

⚠ Segurança

- ❌ **NUNCA** commitar `.env.local` ou `.env.production`
- ☑ Esses arquivos estão no `.gitignore`
- ☑ Use variáveis de ambiente na plataforma de deploy (Vercel, Firebase, etc.)

Scripts Disponíveis

Desenvolvimento

```
# Iniciar servidor de desenvolvimento (porta 3001)
yarn dev

# Acessar: http://localhost:3001
```

Features ativadas:

- ☑ Hot Module Replacement (HMR)
- ☑ Turbopack para builds rápidos
- ⚠ PWA desabilitado (melhor performance)

Build de Produção

```
# Criar build otimizado
yarn build

# Isso gera:
# - Build otimizado em .next/
# - Service Worker compilado em public/sw.js
# - Assets otimizados e minificados
```

Preview de Produção

```
# Testar o build de produção localmente
yarn start

# Acessar: http://localhost:3000
```

Qualidade de Código

```
# Executar ESLint
yarn lint

# Verificar tipos TypeScript
```

```
yarn typecheck

# Executar testes
yarn test

# Testes com cobertura
yarn test:coverage

# Executar tudo (lint + typecheck + test)
yarn check
```

Formatação

```
# Formatar código com Prettier (se configurado)
yarn format
```

Build e Deploy

Vercel (Recomendado)

Vantagens:

- ☑ Deploy automático
- ☑ Preview deployments
- ☑ Suporte nativo a PWA
- ☑ CDN global

Passos:

1. **Instalar Vercel CLI**

   ```
   npm i -g vercel
   ```

2. **Login**

   ```
   vercel login
   ```

3. **Deploy**

   ```
   vercel
   ```

4. **Configurar Variáveis de Ambiente**

    - Acesse o dashboard do Vercel
    - Vá em Settings > Environment Variables
    - Adicione todas as variáveis do `.env.local`

5. **Deploy para Produção**

```
vercel --prod
```

Firebase Hosting

Vantagens:

- ☑ Integração com Firebase services
- ☑ CDN global do Google
- ☑ Free tier generoso

Passos:

1. **Instalar Firebase CLI**

```
npm install -g firebase-tools
```

2. **Login**

```
firebase login
```

3. **Inicializar Hosting**

```
firebase init hosting
```

    - Selecione seu projeto
    - Public directory: `.next` ou `out`
    - Single-page app: **Yes**
    - GitHub deploys: Opcional

4. **Configurar `firebase.json`**

```
{
  "hosting": {
    "public": "out",
    "ignore": ["firebase.json", "**/.*", "**/node_modules/**"],
    "headers": [
      {
```

```
          "source": "/sw.js",
          "headers": [
            {
              "key": "Cache-Control",
              "value": "no-cache"
            },
            {
              "key": "Service-Worker-Allowed",
              "value": "/"
            }
          ]
        }
      ],
      "rewrites": [
        {
          "source": "**",
          "destination": "/index.html"
        }
      ]
    }
  }
```

5. **Build e Deploy**

```
yarn build
firebase deploy --only hosting
```

Netlify

Vantagens:

- ☑ Deploy automático via Git
- ☑ Build previews
- ☑ Free SSL

Passos:

1. **Conectar repositório GitHub**
   - Acesse Netlify
   - "New site from Git"
   - Selecione o repositório
2. **Configurar Build Settings**
   - Build command: `yarn build`
   - Publish directory: `.next`
3. **Adicionar Variáveis de Ambiente**

    - Site settings > Environment variables
    - Adicione todas as variáveis

4. **Deploy**
    - Push para GitHub (deploy automático)

Troubleshooting

Erro: "Module not found"

Causa: Dependências não instaladas

Solução:

```
rm -rf node_modules yarn.lock
yarn install
```

Erro: "Firebase not configured"

Causa: Variáveis de ambiente não configuradas

Solução:

1. Verifique se `.env.local` existe
2. Confirme que todas as variáveis `NEXT_PUBLIC_FIREBASE_*` estão preenchidas
3. Restart o servidor de desenvolvimento

Erro: "Port 3001 is already in use"

Causa: Porta já ocupada

Solução:

```
# Matar processo na porta 3001 (Linux/Mac)
lsof -ti:3001 | xargs kill -9

# Windows
netstat -ano | findstr :3001
taskkill /PID <PID> /F

# Ou use porta diferente
yarn dev -- --port 3002
```

PWA não funciona em desenvolvimento

Causa: PWA está desabilitado em desenvolvimento

Solução: Isso é intencional. Para testar PWA:

```
yarn build
yarn start
```

Erro de TypeScript

Causa: Tipos desatualizados ou conflitos

Solução:

```
# Limpar cache TypeScript
rm -rf .next
yarn typecheck
```

Builds lentos

Causa: Cache corrompido

Solução:

```
# Limpar cache Next.js
rm -rf .next

# Limpar cache Turbopack
rm -rf .turbo

# Rebuild
yarn build
```

Next Steps

Após configuração completa:

1. ☑ Execute `yarn dev` e acesse `http://localhost:3001`
2. ☑ Faça login com Firebase Authentication
3. ☑ Teste features: galeria, participantes, dashboard
4. ☑ Execute `yarn check` antes de commits
5. ☑ Leia Documentação Técnica para entender arquitetura

Recursos Adicionais

- 📖 Next.js Documentation
- 🔥 Firebase Documentation

- 🎨 Tailwind CSS Docs
- 🧩 Radix UI Docs
- 🪢 TanStack Query Docs

← Voltar ao README